Software Design Plan

Version 1 Real-Time YOLO Object Detection Application

Document Status: Software Design Specification
Version: 1.0
Application Type: Local Python Computer-Vision Application
Primary Function: Real-time object detection from a live camera feed

---

1. Purpose

This Software Design Plan defines how Version 1 of the real-time YOLO object detection application will be implemented based on the approved System Architecture Document.

The implementation will provide a complete local application that:

1. Initializes configuration and logging.
2. Opens a local camera.
3. Captures frames continuously.
4. Validates captured frames.
5. Performs required preprocessing.
6. Runs YOLO inference.
7. Converts model-specific results into application-owned detection objects.
8. Validates and filters detections.
9. Draws bounding boxes and labels.
10. Displays the annotated video feed.
11. Detects user shutdown requests.
12. Performs bounded recovery for supported transient failures.
13. Logs application behavior and failures.
14. Performs deterministic cleanup.

The implementation will remain single-process and single-threaded for V1.

---

2. Implementation Objectives

The implementation shall prioritize:

- Clear module boundaries.

- Simple control flow.
- Explicit resource ownership.
- Bounded error recovery.
- Centralized configuration.
- Centralized logging.
- Testability without physical hardware.
- Isolation of YOLO-specific implementation details.
- Predictable startup and shutdown.
- Avoidance of unnecessary runtime complexity.

The implementation shall not introduce:

- Networking.
- Databases.
- Web services.
- Cloud inference.
- Object tracking.
- Object counting.
- Video recording.
- Image persistence.
- Multi-camera processing.
- Custom model training.
- Unnecessary concurrency.

---

3. Technology Stack

The V1 implementation will use:

- Python 3.x.
- OpenCV.
- NumPy where required by image-processing operations.
- Ultralytics YOLO.
- PyTorch or the runtime required by the selected YOLO implementation.
- Python standard-library modules for configuration, logging, timing, error handling, and lifecycle management.

Recommended standard-library modules include:

```
logging
logging.handlers
time
pathlib
dataclasses
enum
typing
traceback
sys
os
contextlib
```

The implementation should avoid additional dependencies unless a documented requirement makes them necessary.

---

4. Project Structure

The implementation should use the following structure:

```
yolo_detection_app/
│
├── main.py
├── config.py
├── camera.py
├── detector.py
├── detection.py
├── preprocessing.py
├── renderer.py
├── logger.py
├── errors.py
│
├── models/
│   └── <pretrained-model-file>
│
├── logs/
│
└── tests/
    ├── test_config.py
    ├── test_camera.py
    ├── test_detector.py
    ├── test_detection.py
    ├── test_preprocessing.py
    ├── test_renderer.py
    ├── test_logging.py
    └── test_application.py
```

The directory structure should remain small unless implementation requirements justify additional modules.

---

5. Module Design

5.1 `main.py`

`main.py` is the process entry point.

Responsibilities:

- Start the application.
- Initialize the application controller.
- Establish the top-level exception boundary.
- Handle `KeyboardInterrupt`.
- Return an appropriate process exit status.

`main.py` should not contain:

- YOLO parsing logic.
- Camera implementation.
- Bounding-box rendering.
- Configuration definitions.
- Detailed recovery algorithms.

Conceptual structure:

```
main()
    |
    +-- initialize application
    |
    +-- run application
    |
    +-- handle shutdown
    |
    +-- return exit status
```

---

6. Application Controller

The application controller may be implemented in `main.py` or in a dedicated `application.py` module.

For a small V1 implementation, keeping it in `main.py` is acceptable provided the module does not become excessively large.

Responsibilities:

- Lifecycle state management.
- Component initialization.

- Main processing loop.
- Failure escalation.
- Shutdown coordination.
- Resource cleanup.

Conceptual methods:

```
start()
run()
request_shutdown(reason)
shutdown()
```

The controller is the only component that should own overall application lifecycle decisions.

Components report failures to the controller rather than directly shutting down the application.

---

7. Application State

An explicit application state representation should be implemented.

Recommended enum:

```
STARTING
INITIALIZING_CAMERA
INITIALIZING_MODEL
RUNNING
RECOVERING
SHUTTING_DOWN
STOPPED
FAILED
```

The state should be stored by the application controller.

Example:

```
STARTING
    |
    v
INITIALIZING_CAMERA
    |
    v
INITIALIZING_MODEL
    |
    v
RUNNING
```

Fatal failures transition toward:

```
FAILED
    |
    v
SHUTTING_DOWN
    |
    v
STOPPED
```

Normal shutdown follows:

```
RUNNING
    |
    v
SHUTTING_DOWN
    |
    v
STOPPED
```

The implementation must prevent normal processing while the application is in `SHUTTING_DOWN`, `STOPPED`, or `FAILED`.

---

8. Configuration Design

8.1 `config.py`

The configuration module contains all operational parameters.

Configuration should preferably be represented using a dataclass or another immutable/simple configuration structure.

Conceptually:

```
Configuration
    camera_index
    camera_width
    camera_height
    camera_fps

    model_path
    model_name
    device

    confidence_threshold
    exit_key
    window_name

    camera_max_read_failures
    camera_retry_delay
    camera_reconnect_attempts
    camera_reconnect_delay

    log_level
    log_directory
    log_file
    log_rotation_policy

    performance_logging_enabled
    performance_log_interval
    debug_detection_logging
```

---

9. Configuration Defaults

Recommended starting values should be defined centrally.

Example:

```
camera_index = 0
camera_width = 640
camera_height = 480
camera_fps = camera-dependent/default

confidence_threshold = 0.50

device = "cpu"

exit_key = "q"

window_name = "YOLO Object Detection"

camera_max_read_failures = bounded small integer
camera_retry_delay = short delay
camera_reconnect_attempts = bounded small integer
camera_reconnect_delay = short delay

log_level = INFO
performance_logging_enabled = True
debug_detection_logging = False
```

Exact retry and performance values should be adjusted during testing.

No operational constant should be duplicated throughout the application.

---

10. Configuration Validation

Configuration validation must occur before normal runtime initialization.

Validation shall include:

```
camera_index >= 0
camera_width > 0
camera_height > 0
confidence_threshold >= 0
confidence_threshold <= 1
retry counts >= 0
retry delays >= 0
model path is not empty
device is supported
log level is valid
exit key is valid
```

Invalid configuration shall result in a controlled startup failure.

The application must not repeatedly retry invalid configuration.

Conceptual flow:

```
Load configuration
      |
```

```
      v
Validate
      |
      +-- valid --> continue
      |
      +-- invalid --> log --> shutdown
```

---

11. Camera Implementation

11.1 `camera.py`

The `Camera` class owns all camera-related operations.

Suggested interface:

```
open()
is_open()
read()
reconnect()
release()
```

Suggested internal state:

```
state
camera_index
capture
consecutive_read_failures
reconnect_attempts
```

The rest of the application should not directly access the OpenCV `VideoCapture` object.

---

12. Camera Initialization

`Camera.open()` shall:

1. Create the OpenCV capture object.
2. Attempt to open the configured camera.
3. Apply supported resolution/FPS configuration.
4. Verify that the camera is actually open.
5. Store the successful capture object.
6. Reset camera failure state.
7. Report success.

Conceptually:

```
create VideoCapture
        |
```

```
        v
isOpened()
   /        \
yes          no
 |            |
 v            v
OPEN       failure
```

A failed initialization must not enter the normal processing loop.

---

13. Camera Read Interface

`Camera.read()` should return an application-level result rather than forcing the application controller to understand OpenCV's raw return format.

Conceptual result:

```
FrameResult
    success
    frame
    reason
```

Successful read:

```
success = True
frame = valid image
```

Failed read:

```
success = False
frame = None
reason = failure description
```

The exact implementation may use a dataclass.

---

14. Camera Read Failure Handling

For each failed camera read:

1. Increment the consecutive failure counter.
2. Log the failure according to the logging policy.
3. Avoid processing invalid frame data.
4. Wait for the configured retry delay.
5. Determine whether the failure threshold has been reached.
6. Attempt recovery when appropriate.

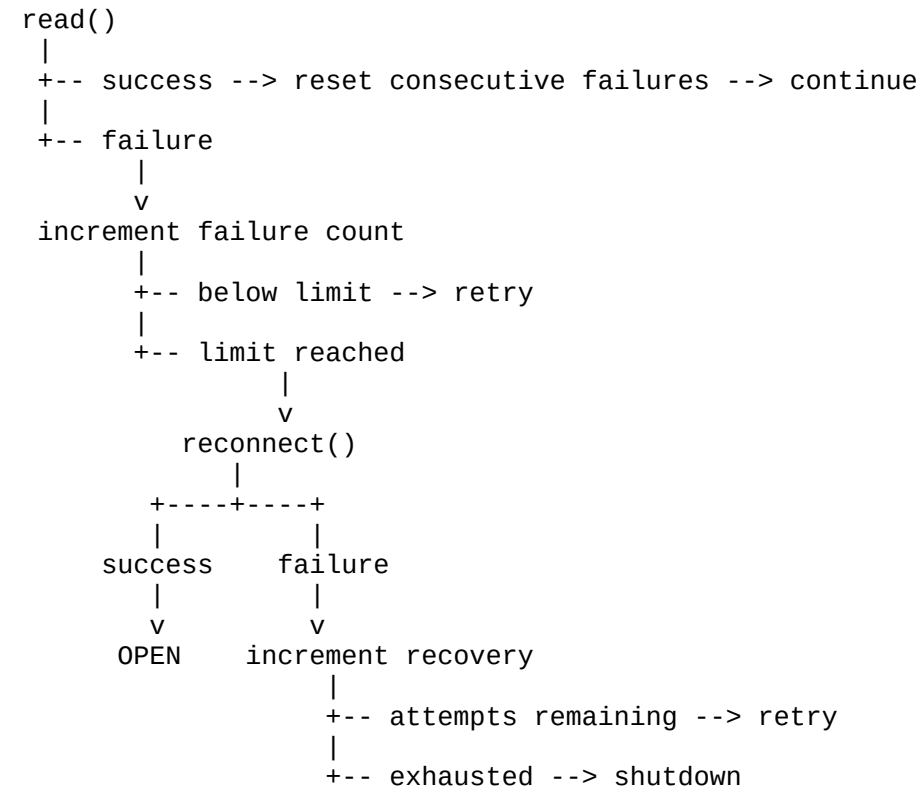
A single transient failure should not automatically terminate the application.

---

15. Camera Recovery Algorithm

The camera recovery process should be bounded.

Conceptually:



The implementation must not reconnect indefinitely.

A successful recovery should reset recovery counters only after the camera has been confirmed operational.

---

16. Camera Release

`Camera.release()` must be safe when:

- The camera was never opened.
- Initialization partially failed.
- The camera was already released.
- A read operation failed.
- Shutdown is called multiple times.

The implementation should check resource state before attempting release.

A release exception should be logged but should not prevent other cleanup operations.

---

17. Frame Validation

17.1 `preprocessing.py`

The module should expose a function similar in purpose to:

```
is_valid_frame(frame)
```

Validation shall check:

```
frame is not None
frame is not empty
frame has expected dimensions
frame has valid image shape
frame has valid dtype
```

The function should return a deterministic result.

Invalid frames should not be passed to the detector.

---

18. Frame Preprocessing

A separate preprocessing stage should exist even if V1 initially performs minimal processing.

Conceptually:

```
preprocess(frame)
    |
    v
processed frame
```

The preprocessing layer should only perform operations required by the model or necessary for safe inference.

It should not contain:

- Detection logic.
- Rendering.
- Camera management.
- Application lifecycle logic.

Unnecessary preprocessing should be avoided because every operation contributes to the frame-processing budget.

---

19. Invalid Frame Behavior

When frame validation fails:

```
Invalid frame
```

```
    |
    v
log diagnostic
    |
    v
skip frame
    |
    v
capture next frame
```

An invalid frame must not:

- Reach YOLO.
- Replace valid runtime state.
- Cause an exception to escape unnecessarily.
- Create an independent infinite retry loop.

If invalid frames continue persistently, the camera subsystem should eventually classify the condition as an input/camera failure.

---

20. Detector Implementation

20.1 `detector.py`

The detector owns the YOLO model.

Suggested interface:

```
load()
is_ready()
detect(frame)
release()
```

Internal state may include:

```
model
device
model_path
consecutive_inference_failures
```

The application must not directly manipulate the model object.

---

21. Model Initialization

The detector initialization sequence is:

```
Validate model configuration
        |
        v
Validate model availability where practical
```

```
        |
        v
Load model
        |
        v
Configure device
        |
        v
Verify detector readiness
        |
        v
READY
```

Model initialization happens before the application enters `RUNNING`.

---

22. Model Loading Failure

Model initialization failure is startup-critical.

Possible causes include:

- Missing model.
- Invalid model path.
- Corrupt model.
- Missing runtime dependency.
- Runtime incompatibility.
- Unsupported model.
- GPU initialization failure.

Behavior:

```
Model failure
    |
    v
log detailed error
    |
    v
do not enter runtime loop
    |
    v
release initialized resources
    |
    v
shutdown
```

Known-invalid configuration must not trigger repeated model-loading attempts.

---

23. Device Selection

The detector should accept a configured device.

Supported conceptual values:

```
cpu
cuda
auto
```

For explicit CPU:

```
device = CPU
```

For explicit CUDA:

```
CUDA available?
   |
   +-- yes --> initialize CUDA
   |
   +-- no --> startup failure
```

For `auto`:

```
supported GPU available?
   |
   +-- yes --> GPU
   |
   +-- no --> CPU
```

The selected device should be logged once during startup.

Silent fallback should not occur when the user explicitly requested a device.

---

24. Inference Interface

The detector should expose:

```
detect(frame)
```

The method should accept an application frame and return an application-owned representation of the detection results.

It must not expose Ultralytics-specific result objects to other modules.

Conceptually:

```
frame
  |
  v
YOLO
  |
  v
native model result
  |
  v
adapter/parser
```

```
  |
  v
Detection[]
```

---

25. Detection Data Model

25.1 `detection.py`

The application should define a `Detection` data structure.

Recommended fields:

```
class_id
class_name
confidence
x1
y1
x2
y2
```

A dataclass is appropriate.

Conceptually:

```
Detection:
    class_id: int
    class_name: str
    confidence: float
    x1: float
    y1: float
    x2: float
    y2: float
```

The representation should be independent of Ultralytics.

---

26. Detection Result Collection

The detector should return:

```
List[Detection]
```

Possible results:

```
[]
```

for no detections, or:

```
[
    Detection(...),
    Detection(...),
    Detection(...)
]
```

for multiple objects.

The application must never assume that exactly one object exists.

---

27. Model Result Conversion

The detector adapter should perform the following operations:

```
Raw YOLO result
      |
      v
Extract boxes
      |
      v
Extract class IDs
      |
      v
Extract confidence
      |
      v
Resolve class names
      |
      v
Create Detection objects
```

Any failure in vendor-specific result parsing should remain within the detector boundary.

---

28. Confidence Filtering

The detection processor applies the configured threshold.

Conceptual logic:

```
for detection in detections:
    if detection.confidence < threshold:
        discard
    else:
        retain
```

Boundary behavior must be deterministic.

Recommended rule:

```
confidence >= threshold
```

means the detection is retained.

A detection exactly equal to the threshold must therefore be accepted.

---

29. Bounding-Box Validation

Each detection must be independently validated.

Required checks:

```
coordinates are numeric
coordinates are finite
x1 <= x2
y1 <= y2
box area > 0
coordinates are compatible with frame size
```

Slightly out-of-bounds coordinates may be clamped.

Conceptually:

```
x1 = max(0, x1)
y1 = max(0, y1)
x2 = min(width, x2)
y2 = min(height, y2)
```

Fundamentally malformed detections should be discarded.

One invalid detection must not automatically discard valid detections from the same frame.

---

30. Class Validation

The detection processor should validate the class ID and class name.

Invalid class information should cause the individual detection to be discarded or converted to a safe representation according to the implementation policy.

The system should not terminate merely because one model result is malformed.

---

31. No-Detection Behavior

An empty detection list is normal.

The expected flow is:

```
Frame
  |
  v
YOLO
  |
  v
[]
  |
  v
Renderer
  |
  v
Display original frame
```

No error should be generated.

DEBUG logging may report zero detections, but this should be sampled or controlled rather than logged on every frame.

---

32. Multiple Detection Behavior

All valid detections returned for the frame should be processed.

Example:

```
person
person
car
bicycle
```

Each detection should independently pass through:

```
validation
    |
confidence filtering
    |
bounding-box validation
    |
rendering
```

V1 does not attempt to associate detections between frames.

---

33. Inference Failure Handling

An inference exception should be caught inside the detector boundary.

Conceptual behavior:

```
detect(frame)
    |
    +-- success --> Detection[]
    |
    +-- exception
            |
            v
       classify failure
            |
            +-- detector still usable --> controlled failure result
            |
            +-- detector unusable --> escalation
```

A single inference error should not automatically destroy the entire application if the detector remains valid.

Repeated inference failures must be bounded.

---

34. Inference Failure Counter

The detector may maintain:

`consecutive_inference_failures`

Successful inference:

`counter = 0`

Failed inference:

`counter += 1`

If the configured threshold is exceeded:

`detector considered unusable`

and the application should transition toward controlled shutdown.

This prevents:

```
inference failure
    |
retry
    |
inference failure
    |
retry
    |
...
```

from continuing indefinitely.

---

35. Renderer Implementation

35.1 `renderer.py`

The renderer owns visual output.

Responsibilities:

- Draw bounding boxes.
- Draw class labels.
- Draw confidence.
- Display frames.
- Process keyboard input.
- Destroy display windows during cleanup.

It should not:

- Load YOLO.
- Read the camera.
- Validate model results.
- Own application lifecycle.

---

36. Rendering Interface

Conceptual interface:

```
render(frame, detections)
show(frame)
exit_requested()
close()
```

The renderer should operate on application-owned `Detection` objects.

It must not need to know how the detector generated them.

---

37. Bounding-Box Rendering

For every valid detection:

```
draw rectangle
draw label
```

Recommended label format:

```
<class_name> <confidence>
```

Examples:

```
person 0.91
car 0.87
dog 0.73
```

Confidence should use consistent formatting.

Two decimal places are recommended for V1.

---

38. Display Processing

The renderer should use OpenCV for:

- Window creation.
- Frame display.
- Keyboard input.

- Window destruction.

The application should avoid introducing another GUI framework for V1.

---

39. Exit-Key Processing

The default exit key should be:

```
Q
```

The value should be configurable.

The renderer should expose a simple result:

```
exit_requested() -> bool
```

The controller decides what to do with that result.

The renderer should not directly call application shutdown.

---

40. Window Closure

If the application can reliably determine that the display window has been closed, it should convert that event into a shutdown request.

Conceptually:

```
Window closed
    |
    v
shutdown requested
    |
    v
controlled cleanup
```

This should not create repeated window-reopen attempts.

---

41. Display Failure

Display exceptions should be caught at the renderer boundary.

If no safe alternate display mechanism exists:

```
Display failure
    |
    v
log error
    |
    v
request shutdown
```

V1 does not require headless-mode recovery.

---

42. Logging Implementation

42.1 `logger.py`

The logger module shall centralize logging configuration.

Responsibilities:

- Create application logger.
- Configure console handler.
- Configure rotating file handler.
- Configure log level.
- Configure formatting.
- Prevent repeated-error flooding.
- Handle logging shutdown.

Modules should acquire their own named logger.

Examples:

```
application
camera
detector
preprocessing
detection
renderer
```

---

43. Logger Initialization

Startup should use a two-stage concept where necessary:

```
minimal logging
    |
    v
configuration loading
    |
    v
full logging configuration
```

If the log directory cannot be initialized, console logging should remain available where possible.

The application must not repeatedly attempt failed logger initialization.

---

44. Log Format

Recommended format:

```
timestamp level component message
```

Example:

```
2026-10-05 00:40:10 INFO camera - Camera 0 opened successfully
2026-10-05 00:40:11 INFO detector - YOLO model loaded
2026-10-05 00:40:11 INFO detector - Using device: cuda
2026-10-05 00:40:12 DEBUG detector - Inference completed in 18.4 ms
2026-10-05 00:40:12 DEBUG detection - 3 valid detections
```

Exceptions should include stack traces where appropriate.

---

45. Logging Levels

INFO

Use for:

```
application startup
configuration accepted
camera opened
model loaded
device selected
recovery succeeded
shutdown requested
shutdown completed
```

DEBUG

Use for:

```
frame validation
frame dimensions
inference timing
detection count
detection data
bounding boxes
state transitions
recovery attempts
performance metrics
```

WARNING

Use for:

```
temporary camera failure
invalid frame
temporary inference failure
recovery attempt
logging degradation
```

ERROR

Use for:

```
camera recovery failure
display failure
model operation failure
file I/O failure
```

CRITICAL

Use for:

```
required startup failure
unusable runtime
fatal application condition
```

---

46. Repeated-Error Suppression

The logging implementation should prevent repeated failures from generating one log message per frame.

A failure tracker should conceptually contain:

```
failure_key
occurrence_count
consecutive_count
first_failure_time
last_failure_time
last_logged_time
suppressed_count
```

Example behavior:

```
First camera failure
    WARNING

Additional failures
    suppressed

Periodic summary
    WARNING:
    camera still failing

Recovery
    INFO:
    camera recovered
```

When the failure condition ends, the logger may report:

```
Camera failure condition ended.
Occurrences: 53.
Suppressed messages: 47.
Duration: 10.2 seconds.
```

---

47. Log Rotation

The file logger should use a rotating handler.

Configuration should specify:

```
maximum file size
number of retained files
```

The implementation must avoid unbounded log growth.

DEBUG-level per-frame logging should not be enabled indefinitely without considering disk usage.

---

48. Logging Failure Handling

If the file logger fails:

```
file handler failure
    |
    v
disable failed handler
    |
    v
emit one warning through surviving console handler
    |
    v
continue if operationally acceptable
```

The logging system must not recursively log through a broken handler.

V1 should treat file logging as non-critical unless deployment requirements explicitly make it mandatory.

---

49. Error Definitions

49.1 `errors.py`

Application-specific exceptions should be defined only where they improve classification.

Recommended types:

```
ApplicationError
ConfigurationError
CameraError
CameraOpenError
CameraReadError
CameraRecoveryError
ModelError
ModelLoadError
InferenceError
RenderingError
```

Not every failure requires a custom exception.

---

50. Exception Policy

Component-level code should catch only exceptions it can meaningfully handle.

For example:

```
Camera
    catches camera-specific exceptions

Detector
    catches model/inference exceptions

Renderer
    catches rendering/display exceptions

Application
    catches final unexpected exceptions
```

The application must not use:

```
except Exception:
    pass
```

as a generic recovery mechanism.

---

51. Top-Level Exception Boundary

The process entry point should provide a final safety boundary.

Conceptually:

```
try:
    initialize
    run
except KeyboardInterrupt:
    controlled shutdown
except ApplicationError:
    log failure
    controlled shutdown
except Exception:
    log traceback
    mark application failed
finally:
    cleanup
```

The broad `Exception` handler exists only as the final safety boundary.

It must not replace component-level error handling.

---

52. Main Processing Loop

The core runtime should follow this sequence:

```
while application is RUNNING:

    check shutdown state

    capture frame

    if capture failed:
        perform bounded recovery
        continue or shutdown

    validate frame

    if frame invalid:
        log
        continue

    preprocess frame

    run YOLO inference

    if inference failed:
        handle failure
        continue or shutdown

    validate detections

    apply confidence threshold

    render detections

    display frame

    process keyboard input

    update performance metrics
```

The loop must not contain substantial subsystem implementation logic.

---

53. Main Loop Pseudocode

Conceptually:

```
initialize application

while not shutdown_requested:

    frame_result = camera.read()

    if not frame_result.success:
        if recovery succeeds:
            continue
        request shutdown
        break

    frame = frame_result.frame
```

```
    if not is_valid_frame(frame):
        log invalid frame
        continue

    processed_frame = preprocess(frame)

    detection_result = detector.detect(processed_frame)

    if detection_result failed:
        handle inference failure
        continue or shutdown

    detections = process_detections(
        detection_result,
        frame dimensions,
        confidence threshold
    )

    display_frame = renderer.render(
        frame,
        detections
    )

    renderer.show(display_frame)

    if renderer.exit_requested():
        request shutdown

    update performance metrics

shutdown()
```

---

54. Processing Order

The implementation should preserve the architectural sequence:

```
Camera
  ->
Frame Validation
  ->
Preprocessing
  ->
Detector
  ->
Detection Processing
  ->
Renderer
  ->
Display
  ->
Input
```

Optimization may alter internal implementation details but must not cause responsibilities to become coupled.

---

55. Performance Instrumentation

Performance timing should use:

```
time.perf_counter()
```

The implementation should measure:

```
capture time
preprocessing time
inference time
detection-processing time
rendering time
total frame time
effective FPS
```

The application should distinguish:

```
inference latency
```

from:

```
end-to-end frame latency
```

---

56. FPS Calculation

The preferred application-level metric is effective processing/display FPS.

A rolling or interval-based calculation should be used instead of relying exclusively on instantaneous frame timing.

Conceptually:

```
frames_processed / elapsed_time
```

Performance logs should be sampled periodically.

They should not generate a log entry on every frame under normal INFO logging.

---

57. Performance Logging

A configurable interval should control performance logging.

Example:

```
performance_log_interval = 5 seconds
```

A periodic performance record may include:

```
FPS
average frame time
inference time
detection count
```

```
skipped frames
camera failures
inference failures
```

The exact interval is configurable.

---

58. CPU and Memory Monitoring

CPU and memory metrics should be sampled periodically rather than every frame.

If a standard-library-only implementation cannot provide suitable metrics, these measurements may be obtained during external performance testing rather than being mandatory in the runtime loop.

Performance monitoring must not materially reduce FPS.

---

59. Resource Ownership

Resources shall be owned by the component that creates them.

```
Camera
    -> VideoCapture

Detector
    -> model/runtime references

Renderer
    -> OpenCV windows

Logger
    -> logging handlers
```

The application controller controls lifecycle order but does not manipulate component internals.

---

60. Startup Implementation

Startup should execute in the following order:

```
1. Process starts.
2. Initialize minimal logging.
3. Load configuration.
4. Validate configuration.
5. Configure complete logging.
6. Initialize application state.
7. Open camera.
8. Verify camera.
9. Load YOLO model.
10. Verify detector.
11. Initialize display.
12. Enter RUNNING.
13. Start main loop.
```

A failure at any stage must trigger cleanup of all resources already initialized.

---

61. Partial Initialization Cleanup

Example:

```
logging initialized
camera initialized
model initialization fails
```

Required behavior:

```
model failure
    |
    v
release camera
    |
    v
close logging
    |
    v
exit
```

Another example:

```
logging initialized
camera initialized
model initialized
display initialization fails
```

Required behavior:

```
display failure
    |
    v
release camera
    |
    v
release model resources
    |
    v
close logging
    |
    v
exit
```

---

62. Shutdown Implementation

Shutdown should be centralized.

Conceptual sequence:

```
request shutdown
    |
    v
```

```
state = SHUTTING_DOWN
    |
    v
stop main loop
    |
    v
camera.release()
    |
    v
renderer.close()
    |
    v
detector.release()
    |
    v
log cleanup status
    |
    v
logger.shutdown()
    |
    v
state = STOPPED
```

---

63. Idempotent Shutdown

Calling shutdown more than once must not cause errors.

Conceptually:

```
if already STOPPED:
    return
```

Each resource release operation should also be safe when the resource was never initialized.

---

64. Cleanup Error Handling

Cleanup operations should be independent.

For example:

```
try camera.release()
catch -> log

try renderer.close()
catch -> log

try detector.release()
catch -> log

try logger.shutdown()
catch -> final fallback
```

A failure to release one resource must not prevent release of other resources.

---

65. Exit Status

The process should return a meaningful status.

Conceptually:

```
0
```

for successful or user-requested shutdown.

A non-zero status should be returned for fatal startup or runtime failures where appropriate.

The exact status-code scheme may remain simple in V1.

---

66. Failure Classification

Each failure should be classified as one of:

```
NORMAL
TRANSIENT
RECOVERABLE
UNRECOVERABLE
PROGRAMMING_ERROR
```

Examples:

```
No detections
    NORMAL

Single camera read failure
    TRANSIENT

Camera disconnect with successful reconnect
    RECOVERABLE

Missing model
    UNRECOVERABLE

Unexpected AttributeError
    PROGRAMMING_ERROR
```

The classification determines whether the application continues, retries, or shuts down.

---

67. Retry Rules

All retry operations must be bounded.

Every retry loop must have:

```
maximum attempts
or
maximum elapsed time
or
explicit escalation condition
```

Examples:

```
Camera reconnect
    bounded attempts

Model loading
    no indefinite retries

Log handler recovery
    no per-frame retries

Display initialization
    no indefinite retries
```

---

68. Dependency Direction

The implementation should follow:

```
main/application
    |
    +-- config
    +-- logger
    +-- camera
    +-- detector
    +-- preprocessing
    +-- detection
    +-- renderer
    +-- errors
```

Lower-level modules must not import the application controller.

For example:

```
camera.py
```

must report:

```
CameraRecoveryError
```

or a structured failure result rather than directly invoking application shutdown.

---

69. Avoiding Circular Dependencies

The following dependency pattern must be avoided:

```
application -> camera
camera -> application
```

Similarly:

```
detector -> renderer
renderer -> detector
```

Components should communicate through their public interfaces.

---

70. Detection Processor Design

The detection processor should expose a function conceptually equivalent to:

```
process_detections(
    raw_detections,
    frame_width,
    frame_height,
    confidence_threshold
)
```

It should:

1. Iterate over model-independent detection objects.
2. Validate each detection.
3. Apply confidence filtering.
4. Validate/clamp coordinates.
5. Return valid detections.

It should not access the camera or display.

---

71. Detection Processing Algorithm

Conceptually:

```
for each detection:

    validate class information

    validate confidence

    if confidence < threshold:
        discard

    validate coordinates

    if coordinates invalid:
        discard

    clamp acceptable out-of-bounds coordinates

    append valid detection
```

The function should not raise an application-fatal exception for a malformed individual model result unless the entire result structure is unusable.

---

72. Renderer Data Contract

The renderer should receive:

```
frame
Detection[]
```

It should not receive:

```
Ultralytics result objects
```

The renderer therefore remains independent of the model implementation.

---

73. Detector Data Contract

The detector should expose:

```
List[Detection]
```

or an equivalent structured result.

The detector owns conversion from:

```
Ultralytics result
```

to:

```
Detection
```

This is one of the most important abstraction boundaries in the implementation.

---

74. Camera Data Contract

The camera should expose an application-level frame result.

Conceptually:

```
FrameResult
    success
    frame
    reason
```

The controller should not need to inspect:

```
cv2.VideoCapture
```

directly.

---

75. Renderer Data Contract

The renderer should expose:

```
render(frame, detections)
show(frame)
exit_requested()
close()
```

The controller decides when these operations are called.

---

76. Detector Lifecycle

The detector lifecycle is:

```
UNINITIALIZED
      |
      v
LOADING
      |
      +-- failure --> FAILED
      |
      v
READY
      |
      +-- runtime failure --> DEGRADED/FAILED
      |
      v
RELEASED
```

A simplified internal representation may be used if the additional states do not improve implementation clarity.

---

77. Camera Lifecycle

The camera lifecycle is:

```
CLOSED
   |
   v
OPENING
   |
   +-- failure --> FAILED
   |
   v
OPEN
   |
   +-- read failure --> RECOVERING
   |                        |
   |                        +-- success --> OPEN
   |                        |
   |                        +-- failure --> FAILED
   |
   v
```

```
RELEASING
   |
   v
CLOSED
```

---

78. Renderer Lifecycle

The renderer may use:

```
UNINITIALIZED
READY
FAILED
CLOSED
```

Display initialization should happen before normal runtime processing.

If the renderer cannot operate, V1 should normally terminate because there is no alternate user interface.

---

79. Performance Budget

The primary target is approximately 30 FPS.

A 30 FPS workload corresponds to:

```
33.3 ms/frame
```

A preferred target is:

```
approximately 25 ms/frame
```

on the defined reference hardware and model.

The implementation should therefore avoid unnecessary operations in:

```
capture
preprocessing
inference
detection processing
rendering
logging
```

---

80. Performance Optimization Priorities

Optimization should occur in this order:

```
1. Measure.
2. Identify bottleneck.
3. Optimize bottleneck.
4. Re-measure.
```

```
5. Verify correctness.
```

Likely bottlenecks include:

```
YOLO inference
frame resolution
CPU/GPU transfer
preprocessing
rendering
excessive logging
```

The application should not introduce multithreading before profiling demonstrates that it is necessary.

---

81. Memory Management Implementation

The processing loop should maintain only the current frame and required temporary objects.

Avoid:

```
global frame history
unbounded lists
unbounded queues
persistent detection history
```

Expected flow:

```
capture
    ->
process
    ->
render
    ->
display
    ->
release transient references
    ->
next frame
```

---

82. Logging Performance

Logging must not become a major component of frame latency.

The following should generally be DEBUG-level and sampled:

```
per-frame FPS
per-frame inference time
per-frame detection count
per-frame detection details
```

INFO logging should contain only significant operational events.

---

83. Unit Test Design

Unit tests must isolate deterministic logic from external resources.

Test areas:

```
configuration
frame validation
detection validation
confidence filtering
bounding-box validation
FPS calculations
failure counters
retry decisions
state transitions
logging behavior
```

No physical camera or GPU should be required for these tests.

---

84. Camera Test Doubles

A fake camera should support scenarios:

```
open succeeds
open fails
read succeeds
read fails once
read fails repeatedly
disconnect occurs
reconnect succeeds
reconnect fails
release succeeds
release raises
```

The test suite must verify:

```
retry limits
counter resets
recovery behavior
shutdown behavior
```

---

85. Detector Test Doubles

A fake detector should simulate:

```
successful model loading
model-loading failure
successful inference
zero detections
multiple detections
malformed detections
single inference failure
repeated inference failures
```

This allows controller behavior to be tested without loading an actual YOLO model.

---

86. Renderer Test Doubles

The renderer should be replaceable with a fake implementation.

The fake renderer should simulate:

```
display succeeds
display fails
exit requested
window closes
cleanup succeeds
cleanup fails
```

This allows lifecycle tests to execute without creating an actual GUI window.

---

87. Time Abstraction

Time-dependent logic such as:

```
retry delay
error rate limiting
FPS measurement
performance intervals
```

should be designed so that tests can control or simulate time.

Where practical, the time source should be isolated behind a small function or interface.

This prevents tests from requiring actual multi-second waits.

---

88. Configuration Tests

Required cases:

```
valid configuration
invalid confidence threshold
invalid camera index
invalid resolution
invalid retry count
invalid retry delay
missing model path
invalid device
invalid logging level
invalid exit key
```

Each invalid case should produce deterministic validation failure.

---

89. Frame Validation Tests

Required cases:

```
None
empty NumPy array
valid BGR frame
wrong dimensionality
invalid dtype
zero-width image
zero-height image
unexpected shape
```

The validation function should never crash because the input itself is malformed.

---

90. Detection Validation Tests

Required cases:

```
valid detection
confidence below threshold
confidence exactly at threshold
confidence above threshold
negative coordinates
reversed coordinates
zero-area box
out-of-bounds box
NaN coordinate
infinite coordinate
invalid class ID
multiple detections
empty detection list
```

Each test should verify the expected acceptance/rejection behavior.

---

91. Camera Recovery Tests

Tests should verify:

```
one failed read -> retry
failure threshold reached -> reconnect
reconnect succeeds -> resume
reconnect fails -> bounded retry
all recovery attempts exhausted -> shutdown
successful recovery -> counters reset
```

A successful recovery must not reset counters prematurely.

---

92. Detector Failure Tests

Tests should verify:

```
single inference failure
recovery/continue behavior
consecutive inference failures
failure threshold
detector unusable state
shutdown escalation
```

The test must ensure that repeated inference errors cannot create an infinite loop.

---

93. Logging Tests

Required logging tests:

```
console initialization
file initialization
invalid log directory
file-handler failure
log level filtering
log rotation
exception traceback
repeated-error suppression
failure summary
logger shutdown
```

Special attention should be given to preventing recursive logging failures.

---

94. Application Lifecycle Tests

Required scenarios:

```
normal startup
camera startup failure
model startup failure
display startup failure
normal Q shutdown
KeyboardInterrupt
unexpected exception
partial initialization
repeated shutdown
cleanup exception
```

Each test should verify the final application state.

---

95. Integration Testing

Integration tests should verify component boundaries:

```
Camera -> preprocessing
preprocessing -> detector
detector -> detection processor
detection processor -> renderer
renderer -> controller
```

Real model inference should be used selectively because it is slower and more environment-dependent than unit tests.

---

96. End-to-End Test

A complete E2E test should verify:

```
application starts
    |
camera opens
    |
model loads
    |
display opens
    |
frames are processed
    |
objects are detected
    |
labels are displayed
    |
multiple detections work
    |
Q is pressed
    |
processing stops
    |
camera released
    |
window destroyed
    |
model resources released
    |
logging closed
    |
process terminates
```

---

97. Fault Injection

The test implementation should deliberately inject failures.

Camera:

```
unavailable
busy
read failure
disconnect
reconnect failure
```

Frames:

```
None
empty
invalid shape
invalid dtype
continuous invalid input
```

Model:

```
missing
corrupt
unsupported
runtime failure
GPU unavailable
```

Inference:

```
single failure
repeated failure
```

Detection:

```
invalid class
invalid confidence
invalid coordinates
```

Display:

```
initialization failure
runtime failure
window closure
```

Logging:

```
directory failure
permission failure
write failure
handler failure
```

Lifecycle:

```
KeyboardInterrupt
unexpected exception
cleanup exception
```

---

98. Performance Test Procedure

Performance qualification should use a documented environment.

The test record should contain:

```
OS
Python version
OpenCV version
Ultralytics version
PyTorch/runtime version
CPU
```

```
GPU
RAM
VRAM
camera
camera resolution
model
model size
device
confidence threshold
test duration
```

Without these parameters, FPS measurements should not be considered reproducible.

---

99. Performance Acceptance Metrics

The performance report should contain:

```
average FPS
minimum FPS
average frame time
95th percentile frame time
maximum frame time
average inference time
maximum inference time
CPU utilization
GPU utilization where applicable
memory usage
VRAM usage where applicable
skipped frames
camera failures
inference failures
startup time
shutdown time
```

---

100. Stress Test

Initial qualification should include:

```
15-minute stability test
60-minute release qualification
```

The 60-minute test should monitor:

```
memory growth
GPU memory growth
FPS degradation
latency growth
camera stability
log growth
resource leaks
window stability
```

A deployment may require longer testing.

---

101. Regression Testing

Every confirmed defect should result in a regression test where practical.

Examples:

```
Camera retry bug
    -> camera recovery regression test

Log flooding
    -> repeated-error suppression test

Cleanup failure
    -> partial initialization cleanup test

Invalid box rendering
    -> bounding-box validation test

FPS measurement bug
    -> performance timing test
```

---

102. Development Phases

Implementation should proceed incrementally.

Phase 1: Project Foundation

Implement:

```
project structure
configuration
basic logging
error definitions
application state
```

Deliverable:

```
application can initialize and validate configuration
```

Phase 2: Camera

Implement:

```
Camera class
open
is_open
read
release
camera state
basic failures
```

Deliverable:

```
application can safely open and display raw camera frames
```

Phase 3: Frame Validation

Implement:

```
frame validation
preprocessing boundary
invalid-frame handling
```

Deliverable:

```
malformed frames do not reach inference
```

Phase 4: YOLO Detector

Implement:

```
model loading
device selection
detector interface
inference
YOLO result conversion
```

Deliverable:

```
application can produce application-owned Detection objects
```

Phase 5: Detection Processing

Implement:

```
confidence filtering
class validation
bounding-box validation
coordinate clamping
multiple detection handling
```

Deliverable:

```
only valid application detections reach renderer
```

Phase 6: Rendering

Implement:

```
bounding boxes
class labels
confidence display
```

```
OpenCV display
exit key
```

Deliverable:

```
complete live annotated detection feed
```

Phase 7: Recovery and Reliability

Implement:

```
camera retry
camera reconnect
inference failure handling
display failure handling
failure counters
rate-limited logging
```

Deliverable:

```
runtime failures are bounded and controlled
```

Phase 8: Lifecycle and Cleanup

Implement:

```
central shutdown
partial initialization cleanup
idempotent release
KeyboardInterrupt
top-level exception handling
```

Deliverable:

```
all normal and abnormal paths terminate cleanly
```

Phase 9: Performance Instrumentation

Implement:

```
frame timing
inference timing
FPS
periodic performance logging
failure counters
```

Deliverable:

```
performance can be measured without materially affecting runtime
```

Phase 10: Testing and Qualification

Implement:

```
unit tests
integration tests
E2E tests
fault injection
performance tests
stress tests
regression tests
```

Deliverable:

```
V1 satisfies functional, reliability, performance, and cleanup criteria
```

---

103. Implementation Order

The recommended implementation dependency order is:

```
errors.py
    |
    v
config.py
    |
    v
logger.py
    |
    v
camera.py
    |
    v
preprocessing.py
    |
    v
detection.py
    |
    v
detector.py
    |
    v
renderer.py
    |
    v
application/main.py
    |
    v
tests
```

This order minimizes integration complexity because foundational modules are implemented before the modules that depend on them.

---

104. Development Milestones

Milestone 1:

```
Configuration + logging operational
```

Milestone 2:

```
Camera feed operational
```

Milestone 3:

```
YOLO inference operational
```

Milestone 4:

```
Detection rendering operational
```

Milestone 5:

```
Complete normal shutdown operational
```

Milestone 6:

```
Camera recovery operational
```

Milestone 7:

```
Failure handling and log suppression operational
```

Milestone 8:

```
Performance instrumentation operational
```

Milestone 9:

```
Automated tests operational
```

Milestone 10:

```
Performance/stress qualification complete
```

---

105. Definition of Done

A component should not be considered complete merely because its normal path works.

Each component must have:

```
normal behavior
error behavior
cleanup behavior
logging behavior
test coverage
```

For runtime components, it should additionally have:

```
bounded recovery behavior
failure escalation behavior
```

---

106. Camera Definition of Done

Camera implementation is complete when:

- Camera opens.
- Camera state is tracked.
- Valid frames are returned.
- Read failures are detected.
- Read retries are bounded.
- Reconnection is bounded.
- Recovery counters reset correctly.
- Camera release is safe.
- Camera behavior can be mocked.
- Failures are logged appropriately.

---

107. Detector Definition of Done

Detector implementation is complete when:

- Model loads successfully.
- Model-loading failures are handled.
- Device selection works.
- Selected device is logged.
- Inference works.
- Model-specific output is isolated.
- `Detection` objects are produced.
- Empty results work.
- Multiple detections work.
- Inference failures are bounded.
- Model resources can be released.

---

108. Detection Processor Definition of Done

Detection processing is complete when:

- Detection objects are validated.
- Confidence filtering works.
- Threshold boundary behavior is correct.
- Class information is validated.
- Bounding boxes are validated.
- Coordinates are clamped when appropriate.
- Malformed detections are discarded safely.
- Multiple detections are preserved.
- Empty detection lists work.

---

109. Renderer Definition of Done

Renderer implementation is complete when:

- Frames can be displayed.
- Bounding boxes are rendered.
- Labels are rendered.
- Confidence is displayed.
- Multiple detections are rendered.
- Empty detections render correctly.
- Exit key is detected.
- Window closure is handled.
- Display failures are escalated.
- Windows are destroyed during cleanup.

---

110. Logging Definition of Done

Logging is complete when:

- Console logging works.
- File logging works.

- Log rotation works.
- Log levels work.
- Component names are included.
- Exceptions contain stack traces.
- Repeated failures are rate-limited.
- Suppressed failures can be summarized.
- File logging failure does not cause recursion.
- Logging shuts down cleanly.

---

111. Application Definition of Done

The application controller is complete when:

- Startup is deterministic.
- Configuration is validated.
- Components initialize in the required order.
- Application states are valid.
- Runtime failures are classified.
- Recovery is bounded.
- Shutdown can be requested.
- KeyboardInterrupt is handled.
- Unexpected exceptions are handled.
- Partial initialization is cleaned up.
- Shutdown is idempotent.
- Correct exit status is returned.

---

112. Security and Privacy Implementation

The implementation must not:

- Upload frames.
- Transmit detections.
- Open network sockets.
- Persist camera frames.

- Store credentials.
- Log raw image data.
- Log environment secrets.

Model paths and other filesystem information should only be logged when diagnostically useful.

---

113. Data Lifecycle

The runtime data lifecycle is:

```
Camera frame
    |
    v
Validation
    |
    v
Preprocessing
    |
    v
YOLO inference
    |
    v
Detection objects
    |
    v
Rendering
    |
    v
Display
    |
    v
Discard
```

No frame history or detection history is maintained.

---

114. No Persistent Detection State

The application should not maintain:

```
previous object IDs
object trajectories
object counts
historical detections
```

because these belong to future tracking/analytics functionality.

Each frame is independently processed.

---

115. V1 Concurrency Design

The implementation must remain single-threaded.

The primary processing sequence is:

```
capture
    ->
validate
    ->
preprocess
    ->
infer
    ->
process
    ->
render
    ->
display
    ->
input
    ->
repeat
```

No frame queue should be introduced.

No background inference worker should be introduced.

No worker thread should be introduced for logging unless profiling later demonstrates a requirement.

---

116. Future Concurrency Boundary

If future performance testing demonstrates that concurrency is necessary, the likely evolution is:

```
Camera
   |
   v
Bounded Frame Queue
   |
   v
Inference Worker
   |
   v
Detection Queue
   |
   v
Renderer
```

The queues must be bounded.

Stale frames may be discarded rather than allowing latency to grow indefinitely.

This is explicitly outside V1 implementation.

---

117. Code Quality Requirements

Implementation code should:

- Follow consistent naming.
- Use type hints where useful.
- Keep functions focused.
- Avoid large monolithic functions.
- Avoid global mutable runtime state.
- Avoid duplicated configuration values.
- Avoid duplicated error-handling logic.
- Use explicit resource ownership.
- Document non-obvious recovery behavior.
- Keep model-specific code isolated.

---

118. Function Size and Responsibility

Functions should perform one logically cohesive task.

Poor design:

```
main_loop()
```

containing:

```
camera initialization
model loading
YOLO parsing
detection validation
rendering
logging setup
cleanup
```

Preferred design:

```
initialize_camera()
initialize_detector()
process_frame()
handle_camera_failure()
process_detections()
shutdown()
```

The controller coordinates these functions.

---

119. Type Safety

Type hints should be used for public interfaces where practical.

Examples:

```
Camera.read() -> FrameResult

Detector.detect(frame) -> list[Detection]

is_valid_frame(frame) -> bool

process_detections(...) -> list[Detection]
```

This makes interfaces easier to understand and test.

---

120. Data Validation Boundary

Validation should occur as close as possible to the point where external/untrusted data enters the application.

Examples:

```
Camera
    -> frame validation

YOLO
    -> result conversion

Detection processor
    -> detection validation

Configuration
    -> configuration validation
```

This prevents malformed data from spreading through the application.

---

121. External Dependency Isolation

The following external systems should be isolated:

```
OpenCV camera
OpenCV display
Ultralytics
PyTorch/CUDA
filesystem
```

The core application should operate primarily on application-owned data structures.

This makes testing and future replacement easier.

---

122. Model Adapter Isolation

All vendor-specific code should remain inside `detector.py`.

For example, code accessing:

```
YOLO result boxes
YOLO class arrays
YOLO confidence arrays
YOLO model names
```

must not appear in:

```
renderer.py
main.py
detection.py
camera.py
```

The rest of the system receives `Detection` objects.

---

123. Error Logging Context

Every operational error should identify its subsystem.

Examples:

```
camera - Camera read failed
detector - Inference failed
renderer - Display failed
configuration - Invalid confidence threshold
```

This allows logs to be diagnosed without reading source code.

---

124. Error Message Requirements

Error messages should contain enough information to diagnose the failure without exposing sensitive information.

Useful:

```
camera index
failure count
operation
configured retry limit
model operation
device
exception type
```

Avoid:

```
raw frame contents
credentials
environment variables
```

```
unnecessary secrets
```

---

125. Recovery Logging

Recovery should produce lifecycle-level logging rather than per-attempt noise.

Example:

```
WARNING:
Camera read failure detected.

INFO:
Beginning camera recovery.

DEBUG:
Camera reconnect attempt 1/3.

DEBUG:
Camera reconnect attempt 2/3.

INFO:
Camera recovered successfully.
```

If recovery fails:

```
ERROR:
Camera recovery failed after 3 attempts.

INFO:
Application shutdown initiated.
```

---

126. Infinite Loop Prevention

The implementation must be reviewed specifically for loops of the form:

```
while True:
    retry()
```

where no termination condition exists.

Every recovery loop must have:

```
attempt counter
or
elapsed-time limit
or
explicit failure transition
```

The only intentionally unbounded loop is the normal runtime processing loop, and it must always have a shutdown path.

---

127. Resource Leak Prevention

The implementation should be reviewed for:

```
camera handles
OpenCV windows
model references
file handles
logging handlers
temporary buffers
```

Every acquired resource must have a corresponding cleanup operation.

The preferred structure is:

```
try:
    startup
    run
finally:
    shutdown
```

with shutdown itself performing defensive checks.

---

128. Testing Without Hardware

The majority of tests should run without:

```
physical camera
GPU
display
network
```

This is achieved through:

```
fake Camera
fake Detector
fake Renderer
controlled Time
temporary filesystem
```

Only hardware-dependent tests should require an actual camera or GPU.

---

129. Release Qualification

Before V1 release, the following should be completed:

```
all unit tests pass
all integration tests pass
E2E test passes
fault injection passes
15-minute stability test passes
60-minute qualification test passes
performance report completed
resource cleanup verified
```

```
log flooding verified
camera recovery verified
model failure verified
display failure verified
```

---

130. Acceptance Test Matrix

Functional acceptance:

```
camera opens
frames captured
YOLO executes
detections produced
class names displayed
confidence displayed
bounding boxes displayed
multiple objects supported
zero detections supported
Q exits application
```

Reliability acceptance:

```
camera open failure -> clean shutdown
camera read failure -> bounded recovery
camera disconnect -> bounded reconnect
model failure -> clean shutdown
inference failure -> controlled handling
invalid frame -> safe skip
display failure -> controlled shutdown
unexpected exception -> cleanup
```

Observability acceptance:

```
startup logged
camera lifecycle logged
model lifecycle logged
device logged
shutdown logged
errors include context
unexpected exceptions include traceback
repeated failures do not flood logs
```

Resource acceptance:

```
camera released
windows destroyed
model resources released where required
log handlers closed
no worker remains active
```

---

131. Traceability

The implementation should maintain a relationship between SAD requirements and implementation/tests.

Conceptually:

```
SAD Requirement
      |
      v
Module/Function
      |
      v
Unit/Integration Test
      |
      v
Acceptance Test
```

Examples:

```
FR: Capture frames
    -> camera.py
    -> test_camera.py
    -> E2E camera test

FR: Detect objects
    -> detector.py
    -> test_detector.py
    -> E2E inference test

FR: Display detections
    -> renderer.py
    -> test_renderer.py
    -> E2E display test

NFR: Bounded recovery
    -> camera.py/application.py
    -> recovery tests
    -> fault-injection test

NFR: No log flooding
    -> logger.py
    -> logging tests
    -> repeated-failure test
```

---

132. Recommended Implementation Sequence

The developer should implement in this order:

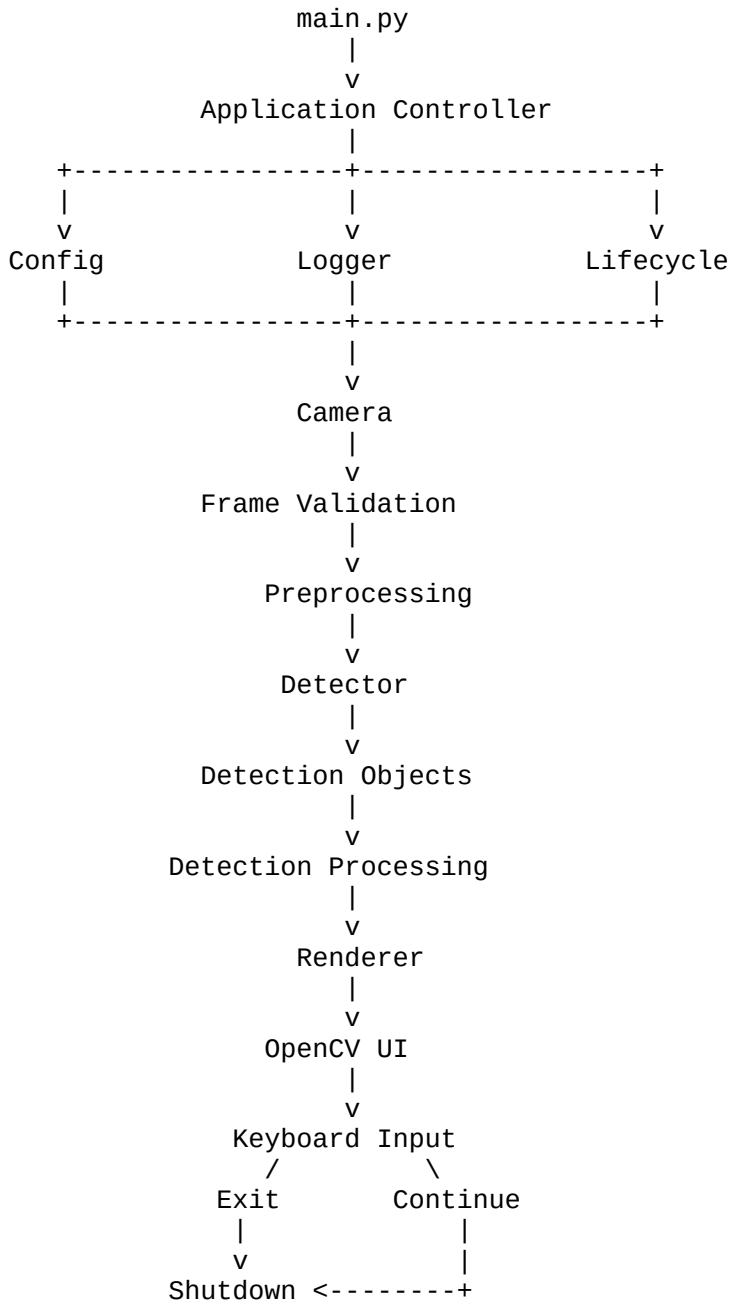
```
1. Create project structure.
2. Implement errors.py.
3. Implement config.py.
4. Implement logger.py.
5. Implement Camera.
6. Implement frame validation.
7. Implement Detection dataclass.
8. Implement detector.
9. Implement detection processing.
10. Implement renderer.
11. Implement application controller.
12. Integrate normal runtime path.
13. Implement camera recovery.
14. Implement inference failure handling.
15. Implement shutdown/cleanup.
```

16. Implement performance instrumentation.
17. Implement unit tests.
18. Implement integration tests.
19. Implement E2E tests.
20. Execute fault testing.
21. Execute performance testing.
22. Execute stress testing.
23. Execute regression suite.
24. Perform final acceptance review.

---

133. Final Software Design

The resulting V1 implementation should have the following conceptual structure:



---

134. Final Runtime Algorithm

The V1 runtime algorithm is:

```
START

initialize logging

load configuration

validate configuration

initialize application state

open camera

if camera initialization fails:
    cleanup
    exit

load YOLO model

if model initialization fails:
    cleanup camera
    exit

initialize renderer

if renderer initialization fails:
    cleanup
    exit

set application state to RUNNING

while application state is RUNNING:

    start frame timer

    read camera frame

    if read failed:
        handle bounded camera recovery

        if recovery succeeds:
            continue

        request shutdown
        break

    validate frame

    if invalid:
        record diagnostic
        continue

    preprocess frame

    run detector

    if inference fails:
        handle inference failure

        if detector remains usable:
```

```
            continue

        request shutdown
        break

    validate detections

    apply confidence filtering

    validate/clamp bounding boxes

    render detections

    display frame

    check exit key

    if exit requested:
        request shutdown

    update performance metrics

shutdown

release camera

destroy windows

release model/runtime resources

close logging

return process status

END
```

---

135. Final Architectural Constraints

The implementation shall maintain the following constraints throughout development:

1. The application controller owns lifecycle decisions.
2. Camera access is isolated in `camera.py`.
3. YOLO-specific behavior is isolated in `detector.py`.
4. Detection data is represented by application-owned structures.
5. Rendering does not perform inference.
6. Configuration is centralized.
7. Logging is centralized.
8. Runtime state is separate from configuration.
9. Recovery is bounded.
10. Repeated failures are rate-limited or aggregated.

11. Invalid frames do not reach inference.
12. Invalid detections do not reach rendering.
13. No frame history is retained.
14. No detection history is retained.
15. No networking is introduced.
16. No database is introduced.
17. No unnecessary threads are introduced.
18. Cleanup is performed on every exit path.
19. Cleanup is safe after partial initialization.
20. Unexpected exceptions remain visible through stack traces.
21. Logging failures cannot create recursive logging loops.
22. Performance instrumentation must remain lightweight.
23. Model-specific result objects must not propagate outside the detector boundary.
24. V1 functionality must remain limited to real-time local object detection.

---

136. V1 Completion Criteria

V1 is complete when the implementation can demonstrate all of the following:

Functional

```
[PASS] Configuration initializes
[PASS] Camera opens
[PASS] Live feed is captured
[PASS] Frames are validated
[PASS] YOLO model loads
[PASS] Correct device is selected
[PASS] Inference executes
[PASS] Detection objects are generated
[PASS] Confidence filtering works
[PASS] Bounding boxes are validated
[PASS] Multiple detections work
[PASS] Zero detections work
[PASS] Labels render
[PASS] Bounding boxes render
[PASS] Live display works
[PASS] Q requests shutdown
```

Reliability

```
[PASS] Camera startup failure is bounded
[PASS] Camera read failures are bounded
[PASS] Camera reconnect is bounded
[PASS] Recovery counters reset correctly
```

[PASS] Model failure causes controlled shutdown
[PASS] Inference failure is controlled
[PASS] Invalid frames are skipped
[PASS] Invalid detections are discarded
[PASS] Display failure is controlled
[PASS] Unexpected exceptions are caught at top level

Logging

[PASS] Startup is logged
[PASS] Camera lifecycle is logged
[PASS] Model lifecycle is logged
[PASS] Device selection is logged
[PASS] Failures contain component context
[PASS] Exceptions contain tracebacks
[PASS] Repeated failures are rate-limited
[PASS] Log files rotate
[PASS] Logging failures do not recurse

Resource Management

[PASS] Camera is released
[PASS] Windows are destroyed
[PASS] Model resources are released where required
[PASS] Log handlers are closed
[PASS] Partial initialization is cleaned up
[PASS] Repeated shutdown is safe

Performance

[PASS] FPS is measurable
[PASS] Frame latency is measurable
[PASS] Inference latency is measurable
[PASS] Performance logging is sampled
[PASS] Reference workload is documented
[PASS] Approximately 30 FPS target is evaluated
[PASS] 95th-percentile frame time is recorded
[PASS] Sustained operation is tested

Testing

[PASS] Unit tests pass
[PASS] Camera tests pass
[PASS] Detector tests pass
[PASS] Detection tests pass
[PASS] Renderer tests pass
[PASS] Logging tests pass
[PASS] Lifecycle tests pass
[PASS] Integration tests pass
[PASS] E2E test passes
[PASS] Fault-injection tests pass
[PASS] Performance tests pass
[PASS] Stress tests pass
[PASS] Regression suite passes

---

137. Final Software Design Position

The V1 implementation should remain a simple local Python application with a sequential real-time pipeline:

```
Camera
  ->
Frame Validation
  ->
Preprocessing
  ->
YOLO Inference
  ->
Detection Processing
  ->
Rendering
  ->
Display
  ->
Repeat
```

The implementation complexity should be concentrated at the boundaries where it provides real engineering value:

```
configuration
lifecycle management
failure classification
bounded recovery
logging
resource cleanup
testing
performance measurement
```

The application should not solve future problems prematurely. Tracking, counting, recording, multiple cameras, networking, databases, cloud inference, GUI frameworks, and distributed processing remain outside V1.

The resulting implementation should nevertheless provide clean interfaces for these future capabilities.

The most important implementation boundary is:

```
YOLO-specific result
        |
        v
Detector Adapter
        |
        v
Application-owned Detection[]
        |
        v
Detection Processing
        |
        v
Renderer
```

This prevents the initial YOLO implementation from becoming embedded throughout the application.

The final V1 product is therefore a complete, testable, fault-tolerant local object-detection application rather than a prototype that merely demonstrates YOLO inference.